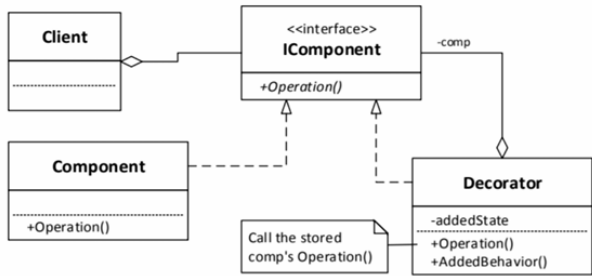


1) Decorator Pattern ជាអ្វី?

Decorator Pattern គឺជា pattern មួយសម្រាប់ បន្ថែម state ឬ behavior ទៅលើ object មួយជាលក្ខណៈ dynamic នៅពេល run time ដោយ មិនចំ: ពាល់ដល់ object ដើម។ វាសមស្របពេលយើងចង់បន្ថែមមុខងារថ្មីទៅ object មួយ ដោយមិនចង់កែប្រែ class ដើម ឬបង្កើត subclass ច្រើនពេក។



Design Patterns មាន ៣ ប្រភេទសំខាន់ៗ:

1. Creational Design Patterns

ជាប្រភេទ pattern ដែលមានគោលបំណងសម្រាប់ ការបង្កើត Object ឬ Class Instantiation។ វាជួយឱ្យការបង្កើត object មានលក្ខណៈបត់បែន ងាយគ្រប់គ្រង និងកាត់បន្ថយភាពស្មុគស្មាញក្នុងការបង្កើត object។
ឧទាហរណ៍: Singleton, Factory Method, Builder, Prototype។

2. Structural Design Patterns

ជាប្រភេទ pattern ដែលមានគោលបំណងសម្រាប់ រៀបចំទំនាក់ទំនងរវាង classes និង objects ឱ្យមានរចនាសម្ព័ន្ធផ្សេងៗ និងបត់បែន។ វាជួយឱ្យផ្នែកផ្សេងៗនៃ system អាចធ្វើការជាមួយគ្នាបាន ទោះបីមានការផ្លាស់ប្តូរផ្នែកខ្លះក៏ដោយ ហើយក៏អាចបន្ថែម functionality ថ្មីទៅ object បានផងដែរ។
ឧទាហរណ៍: Decorator, Adapter, Bridge, Composite។

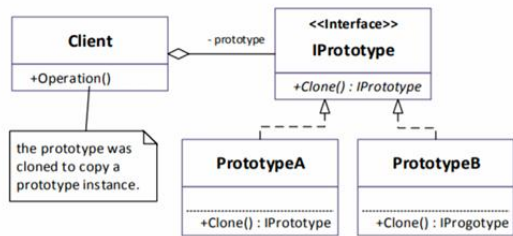
3. Behavioral Design Patterns

ជាប្រភេទ pattern ដែលមានគោលបំណងសម្រាប់ កំណត់ទំនាក់ទំនង និងការប្រតិបត្តិការរវាង objects។ វាផ្តោតលើរបៀបដែល objects ទាក់ទងគ្នា សហការគ្នា និងដោះស្រាយបញ្ហាផ្សេងៗតាម algorithm ឬ behavior ជាក់លាក់។
ឧទាហរណ៍: Strategy, State, Observer, Command, Chain of Responsibility, Template Method។

2) Prototype Pattern មានអត្ថប្រយោជន៍អ្វីខ្លះ?

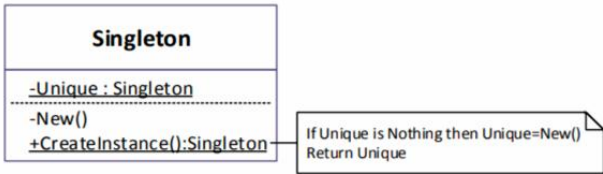
Prototype Pattern គឺជាវិធីបង្កើត object ថ្មី ដោយ clone ពី object ដែលមានស្រាប់។ អត្ថប្រយោជន៍របស់វា គឺអាចលក់ concrete classes ពី

client, អាចបន្ថែម ឬលុបការចម្លងថ្មីនៅពេល runtime, កាត់បន្ថយចំនួន classes នៅក្នុង system និងអាចសម្របតាមការផ្លាស់ប្តូររចនាសម្ព័ន្ធនានាបានយ៉ាងងាយនៅពេល runtime បាន។



3) Singleton Pattern ជាអ្វី?

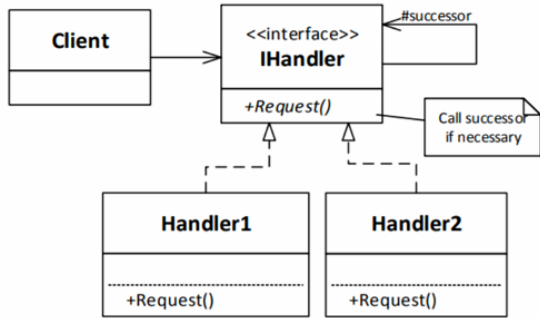
Singleton Pattern គឺជា pattern ដែលធានាថា Class មួយអាចមាន instance តែមួយប៉ុណ្ណោះ ហើយមានលក្ខណៈជា global access point សម្រាប់ប្រើ object នោះ។ វាសមស្របពេលយើងត្រូវការគ្រប់គ្រង object មួយតែមួយក្នុង system ទាំងមូល ដូចជា database connection ឬ configuration manager។



4) Chain of Responsibility Pattern ជាអ្វី? ឧទាហរណ៍ Real World?

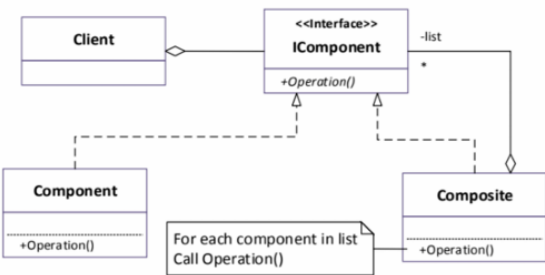
Chain of Responsibility Pattern គឺជា pattern ដែលរៀប handler objects ជា ខ្សែបន្តគ្នា (chain) ដើម្បីទទួល request។ បើ handler មួយមិនអាចដោះស្រាយបាន វានឹងបញ្ជូន request ទៅ handler បន្ទាប់។ ឧទាហរណ៍ក្នុងជីវិតពិត គឺដូចជា ប្រព័ន្ធអនុវត្តចរាចរណ៍: បុគ្គលិក → ប្រធានផ្នែក → អ្នក

គ្រប់គ្រង → នាយក។ អ្នកណាអាចសម្រេចបាន ក៏ដោះស្រាយនៅទីនោះ។



5) Composite Pattern ជាអ្វី?

Composite Pattern គឺជា pattern រៀបចំ object និង object ឱ្យទៅជាជួរឈរសម្ព័ន្ធនៃ tree ដូចជា Single Item និង Group of Items។ គោលបំណងរបស់វាគឺឱ្យ client អាច treat individual objects និង groups of objects ដូចគ្នា បាន។ ឧទាហរណ៍: file មួយ និង folder មួយដែលមាន file ច្រើននៅខាងក្នុង។

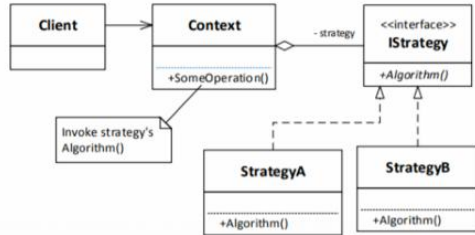


6) Strategy Pattern ជាអ្វី?

Strategy Pattern គឺជា pattern ដែល ជក់ algorithm ចេញពី host class ហើយដាក់ទៅក្នុង classes ផ្សេងៗ ដើម្បីឱ្យ client អាច ជ្រើសរើស algorithm ដែលត្រូវប្រើបាន។ វាសមស្របពេលមាន algorithms ច្រើនដែលធ្វើការដោយស្រដៀងគ្នា ប៉ុន្តែវិធីដោះស្រាយខុសគ្នា ដូចជា sort data, compress

file, ឬជ្រើសរើស transportation/travelling method។

• ជ្រើសរើស strategy ត្រឹមត្រូវ



7) Adapter Pattern ជាអ្វី?

Adapter Pattern គឺជា pattern សម្រាប់ សម្រប interface មិនត្រូវគ្នា រវាង classes ឬ systems ផ្សេងៗ ឱ្យអាចធ្វើការជាមួយគ្នាបាន។ វាសមស្របពេល class មួយមាន interface មិនត្រូវនឹងអ្វីដែល system ត្រូវការ ហើយយើងចង់ reuse code ចាស់ ដោយមិនបាច់សរសេរថ្មីទាំងអស់។ ឧទាហរណ៍: បង្កើន connector ឬ interface រវាង system ចាស់ និង system ថ្មី។

